

Homework 3: LLM-as-Judge for Recipe Bot Evaluation

Your Core Task: Evaluate "Adherence to Dietary Preferences" for the Recipe Bot

Example: If a user asks for a "vegan" recipe, does the bot provide one that is actually vegan?

We'll provide ~2000 starter Recipe Bot traces and detailed criteria for this failure mode if you want a head start! We picked this criterion because it's fairly easy to align an LLM judge on dietary restrictions - the rules are generally clear and objective.

Alternatively, you can choose a different Recipe Bot failure mode you identified in HW2/Chapter 3, but you'll need to define criteria and generate/source all your own traces.

Tools You'll Use

- Your preferred LLM (for crafting the judge)
- Your critical thinking & prompt engineering skills!
- The **judgy** Python library: github.com/ai-evals-course/judgy

Three Implementation Options

You have three options for how much of the pipeline to implement yourself:

Option 1: Full Implementation (Most Learning)

Start from scratch and implement the complete pipeline:

- Generate your own Recipe Bot traces
- Label your own data
- Build the entire evaluation workflow
- **Learning:** Complete end-to-end experience with LLM-as-Judge methodology

Option 2: Start with Raw Traces (Medium Implementation)

Use our provided **raw_traces.csv** (~2400 traces) and focus on the evaluation:

- Skip trace generation, start with labeling
- Implement judge development and evaluation
- Focus on the core LLM-as-Judge workflow
- **Learning:** Judge development, bias correction, and statistical evaluation
- **Note:** Our traces were generated using **dietary_queries.csv** - 60 moderate to challenging edge queries we crafted

Option 3: Start with Labeled Data (Judge Development Focus)

Use our provided **labeled_traces.csv** (101 labeled examples):

- Skip trace generation and labeling

- Focus on judge prompt engineering and evaluation
- Implement the statistical correction workflow
- **Learning:** Judge optimization and bias correction techniques

Choose the option that best fits your learning goals and available time!

Assignment Steps: From Labels to Confident Measurement

Step 1: Get & Label Your Data (Crucial!) *[Option 1 & 2 start here]*

- **If using our "Dietary Adherence" task:** Manually label a subset of the provided traces (e.g., 100-200 examples) as "Pass" or "Fail" based on the provided criteria. This is your ground truth!
- **If using your own failure mode:** Generate/collect and label your traces.
- **Option 1:** Generate your own traces first, then label them. Feel free to use `data/dietary_queries.csv` as a starting point for queries to generate traces with.
- **Option 2:** Use our provided `raw_traces.csv`, then label a subset

Step 2: Split Your Labeled Data *[Option 3 starts here]*

- Divide your labeled set into Train (~10-20%), Dev (~40%), and Test (~40-50%)
- **Option 3:** Use our provided `labeled_traces.csv` and split it

Step 3: Develop Your LLM-as-Judge Prompt *[All options continue from here]*

Craft a clear prompt with:

- The specific task/criterion
- Precise Pass/Fail definitions
- 2-3 clear few-shot examples (input, Recipe Bot output, desired judge reasoning & Pass/Fail label) taken from your Train set
- The structured output format you expect from the judge (e.g., JSON with reasoning and answer)

Step 4: Refine & Validate Your Judge

- Iteratively test and refine your judge prompt using your Dev set
- Measure and report your judge's TPR & TNR on the Dev set during refinement
- Once finalized, report the judge's final TPR & TNR on your Test set

Step 5: Measure on "New" Traces

- Run your finalized judge over a larger set of "new" Recipe Bot traces (e.g., 500-1000 more from the provided set, or newly generated ones)
- This simulates evaluating production data

Step 6: Report Results with judgy

Report:

- The raw pass rate (p_{obs}) from your judge on the new traces
- The corrected true success rate (θ)
- The 95% Confidence Interval (CI) for θ

- Include a brief interpretation of your results (e.g., How well is the Recipe Bot adhering to dietary preferences? How confident are you in this assessment?)

Failure Mode: Adherence to Dietary Preferences

Definition: When a user requests a recipe with specific dietary restrictions or preferences, the Recipe Bot should provide a recipe that actually meets those restrictions and preferences.

Examples:

-  Pass: User asks for "vegan pasta recipe" → Bot provides pasta with nutritional yeast instead of parmesan
-  Fail: User asks for "vegan pasta recipe" → Bot suggests using honey as a sweetener (honey isn't vegan)
-  Pass: User asks for "gluten-free bread" → Bot provides recipe using almond flour and xanthan gum
-  Fail: User asks for "gluten-free bread" → Bot suggests using regular soy sauce (contains wheat) in the recipe
-  Pass: User asks for "keto dinner" → Bot provides cauliflower rice with high-fat protein
-  Fail: User asks for "keto dinner" → Bot includes sweet potato as a "healthy carb" (too high-carb for keto)

Dietary Restriction Definitions (for reference; taken from OpenAI o4):

- **Vegan:** No animal products (meat, dairy, eggs, honey, etc.)
- **Vegetarian:** No meat or fish, but dairy and eggs are allowed
- **Gluten-free:** No wheat, barley, rye, or other gluten-containing grains
- **Dairy-free:** No milk, cheese, butter, yogurt, or other dairy products
- **Keto:** Very low carb (typically <20g net carbs), high fat, moderate protein
- **Paleo:** No grains, legumes, dairy, refined sugar, or processed foods
- **Pescatarian:** No meat except fish and seafood
- **Kosher:** Follows Jewish dietary laws (no pork, shellfish, mixing meat/dairy)
- **Halal:** Follows Islamic dietary laws (no pork, alcohol, proper slaughter)
- **Nut-free:** No tree nuts or peanuts
- **Low-carb:** Significantly reduced carbohydrates (typically <50g per day)
- **Sugar-free:** No added sugars or high-sugar ingredients
- **Raw vegan:** Vegan foods not heated above 118°F (48°C)
- **Whole30:** No grains, dairy, legumes, sugar, alcohol, or processed foods
- **Diabetic-friendly:** Low glycemic index, controlled carbohydrates
- **Low-sodium:** Reduced sodium content for heart health

Sample Challenging Queries

Contradictory Requests:

- "I'm vegan but I really want to make something with honey - is there a good substitute?"
- "I want a cheeseburger but I'm dairy-free and vegetarian"

Ambiguous Preferences:

- "Something not too carb-y for dinner"
- "Something keto-ish but not super strict"
- "Dairy-free but cheese is okay sometimes"

Key Metrics to Understand

- **True Positive Rate (TPR)**: How often the judge correctly identifies adherent recipes
- **True Negative Rate (TNR)**: How often the judge correctly identifies non-adherent recipes
- **Corrected Success Rate**: True adherence rate accounting for judge errors
- **95% Confidence Interval**: Range for the corrected success rate

Deliverables

1. **Your labeled dataset** with train/dev/test splits
2. **Your final judge prompt** with few-shot examples
3. **Judge performance metrics** (TPR/TNR on test set)
4. **Final evaluation results** using judgy (raw rate, corrected rate, confidence interval)
5. **Brief analysis** (1-2 paragraphs) interpreting your results

Reference Implementation

This repository contains a complete reference implementation showing one approach to this assignment. You can:

- **Study the code structure** to understand the workflow
- **Use our provided data** as a starting point
- **Implement your own version** from scratch for full learning value

The reference implementation uses automated labeling with GPT-4o for demonstration purposes. **In practice, you should manually review and correct all labels** for reliable evaluation.

Reference Implementation Structure

```
homeworks/hw3/
├── scripts/
│   ├── generate_traces.py           # Generate Recipe Bot traces with
parallel processing
│   ├── label_data.py               # Use GPT-4o to label ground truth
(150 examples)
│   ├── split_data.py               # Split data into train/dev/test sets
│   └── develop_judge.py            # Develop LLM judge with few-shot
examples
│   └── evaluate_judge.py           # Evaluate judge performance on test
set
├── run_full_evaluation.py           # Run judge on all traces and compute
metrics
├── data/
│   ├── dietary_queries.csv         # 60 challenging edge case queries we
crafted
│   └── raw_traces.csv              # Generated Recipe Bot traces (~2400
```

```
total)
|   |— labeled_traces.csv          # Traces with ground truth labels
(150)
|   |— train_set.csv              # Training examples for few-shot (~23)
|   |— dev_set.csv               # Development set for judge refinement
(~60)
|   |— test_set.csv              # Test set for final evaluation (~67)
|— results/
|   |— judge_performance.json     # TPR/TNR metrics on test set
|   |— final_evaluation.json     # Results with confidence intervals
|   |— judge_prompt.txt          # Final judge prompt
```

How to Run the Reference Implementation

```
# From project root directory
cd homeworks/hw3

# Step 1: Generate traces (creates raw_traces.csv)
python scripts/generate_traces.py

# Step 2: Label data (creates labeled_traces.csv)
python scripts/label_data.py

# Step 3: Split data (creates train/dev/test sets)
python scripts/split_data.py

# Step 4: Develop judge (creates judge_prompt.txt)
python scripts/develop_judge.py

# Step 5: Evaluate judge (creates judge_performance.json)
python scripts/evaluate_judge.py

# Step 6: Final evaluation (creates final_evaluation.json)
python scripts/run_full_evaluation.py
```

Our Final Results

Here were our final results from running the complete reference implementation:

```
Raw Observed Success Rate: 0.857 (85.7%)
Corrected Success Rate: 0.926 (92.6%)
95% Confidence Interval: [0.817, 1.000]
                        [81.7%, 100.0%]
Correction Applied: 0.069 (6.9 percentage points)
```

This suggests the Recipe Bot has strong dietary adherence (92.6% corrected success rate), with the judge initially underestimating performance due to false negatives. The 6.9 percentage point correction indicates

our judge had some bias that was successfully accounted for using the judgy library.

Setup

1. Install dependencies: `pip install -r requirements.txt` (from project root)
2. Configure your LLM API keys in `.env`
3. Choose your failure mode and begin labeling!

Good luck with your evaluation!